

C++ Standard Library Immediate Issues to be moved in Kona, Nov. 2022

Doc. no. P2704R0

Date: 2022-11-11

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Immediate Issues

[3515](#). §[stacktrace.basic.nonmem]: operator<< should be less templated

Section: 19.6.2 [\[stacktrace.syn\]](#), 19.6.4.6 [\[stacktrace.basic.nonmem\]](#) **Status:** [Immediate](#) **Submitter:** Jiang An **Opened:** 2021-01-25 **Last modified:** 2022-11-09

Priority: 2

View other [active issues](#) in [stacktrace.syn].

View all other [issues](#) in [stacktrace.syn].

Discussion:

According to 23.4.4.4 [\[string.io\]](#), the operator<< overloads in 19.6.4.6 [\[stacktrace.basic.nonmem\]](#) are well-formed only if the template parameters charT and traits are char and std::char_traits<char> (that of std::string) respectively, because it is required in *Effects*: that these overloads behave as-if insert a std::string.

I think the declarations of these overloads should be changed to:

```
ostream& operator<<(ostream& os, const stacktrace_entry& f);

template<class Allocator>
ostream& operator<<(ostream& os, const basic_stacktrace<Allocator>& st);
```

[2021-03-12; Reflector poll]

Set priority to 2 and status to LEWG following reflector poll.

[2022-11-07; Kona]

Move to Immediate.

Proposed resolution:

This wording is relative to [N4878](#).

1. Modify 19.6.2 [\[stacktrace.syn\]](#), header <stacktrace> synopsis, as indicated:

```
namespace std {
    // 19.6.3 \[stacktrace.entry\], class stacktrace_entry
    class stacktrace_entry;

    // 19.6.4 \[stacktrace.basic\], class template basic_stacktrace
    template<class Allocator>
    class basic_stacktrace;

    [...]

    // 19.6.4.6 \[stacktrace.basic.nonmem\], non-member functions
    [...]

    string to_string(const stacktrace_entry& f);

    template<class Allocator>
```

```

        string to_string(const basic_stacktrace<Allocator>& st);

template<class charT, class traits>
basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const stacktrace_entry& f);

template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);

    [...]
}

```

2. Modify 19.6.4.6 [\[stacktrace.basic.nonmem\]](#) as indicated:

```

template<class charT, class traits>
basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const stacktrace_entry& f);

-4- Effects: Equivalent to: return os << to_string(f);

template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);

-5- Effects: Equivalent to: return os << to_string(st);

```

3569. join_view fails to support ranges of ranges with non-default_initializable iterators

Section: 26.7.14.3 [\[range.join.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2021-06-16 **Last modified:** 2022-11-11

Priority: 3

View other [active issues](#) in [\[range.join.iterator\]](#).

View all other [issues](#) in [\[range.join.iterator\]](#).

Discussion:

`join_view::iterator` has exposition-only members `outer_` — which holds an iterator into the adapted range — and `inner_` — which holds an iterator into the range denoted by `outer_`. After application of [P2325R3](#) "Views should not be required to be default constructible" to the working draft, single-pass iterators can be non-`default_initializable`. P2325R3 constrains `join_view::iterator`'s default constructor to require that the types of both `outer_` and `inner_` are `default_initializable`, indicating an intent to support such iterator types. However, the effect of the non-default constructor specified in 26.7.14.3 [\[range.join.iterator\]](#) paragraph 6 is to default-initialize `inner_`, which is ill-formed if its type is not `default_initializable`.

[2021-06-23; Reflector poll]

Set priority to 3 after reflector poll.

Previous resolution [SUPERSEDED]:

Wording relative to the [post 2021-06 virtual plenary working draft](#). This PR is currently being implemented in MSVC.

1. Modify 26.7.14.3 [\[range.join.iterator\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>> &&
            (is_reference_v<range_reference_t<V>> ||
             view<range_value_t<V>>)
        template<bool Const>
        struct join_view<V>::iterator {
            [...]
            optional<InnerIter> inner_ = InnerIter{};
            [...]
            constexpr decltype(auto) operator*() const { return *inner_; }
            [...]
            friend constexpr decltype(auto) iter_move(const iterator& i)
            noexcept(noexcept(ranges::iter_move(*i.inner_))) {
                return ranges::iter_move(*i.inner_);
            }
        }
    }
}

```

```

        friend constexpr void iter_swap(const iterator& x, const iterator& y)
            noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
            requires indirectly_swappable<InnerIter>;
    };
}

[...]
```

constexpr void satisfy(); // exposition only

-5- *Effects*: Equivalent to:

```

    auto update_inner = [this](const iterator_t<Base>& x) -> auto&& {
    [...]
    };
    for (; outer_ != ranges::end(parent_>base_); ++outer_) {
        auto&& inner = update_inner(*outer_);
        inner_ = ranges::begin(inner);
        if (*inner_ != ranges::end(inner))
            return;
    }
    if constexpr (ref-is-glvalue)
        inner_.reset().reset();

```

[...]

```

constexpr InnerIter operator->() const
    requires has_arrow<InnerIter> && copyable<InnerIter>;

```

-8- *Effects*: Equivalent to: return *inner_;

```

constexpr iterator& operator++();

```

-9- Let inner-range be:

[...]

-10- *Effects*: Equivalent to:

```

    auto&& inner_rng = inner-range;
    if (++*inner_ == ranges::end(inner_rng)) {
        ++outer_;
        satisfy();
    }
    return *this;

```

[...]

```

constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;

```

-13- *Effects*: Equivalent to:

```

    if (outer_ == ranges::end(parent_>base_))
        inner_ = ranges::end(*--outer_);
    while (*inner_ == ranges::begin(*outer_))
        *inner_ = ranges::end(*--outer_);
    --*inner_;
    return *this;

```

[...]

```

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
    requires indirectly_swappable<InnerIter>;

```

-16- *Effects*: Equivalent to: return ranges::iter_swap(*x.inner_, *y.inner_);

[2021-08-23; Louis Dionne comments and provides improved wording]

I believe the currently proposed resolution is missing the removal of the default_initializable<InnerIter> constraint on join_view::iterator's default constructor in 26.7.14.3 [\[range.join.iterator\]](#). Indeed, after the currently-proposed resolution, join_view::iterator reads like:

```

template<input_range V>
    requires [...]
struct join_view<V>::iterator {
private:
    optional<InnerIter> inner_; // exposition only
    [...]
public:
    iterator() requires default_initializable<OuterIter> &&
        default_initializable<InnerIter> = default;
    [...]
};

```

I believe we should drop the `default_initializable<InnerIter>` constraint from the default constructor (that seems like an oversight unless I missed something):

```

template<input_range V>
    requires [...]
struct join_view<V>::iterator {
private:
    optional<InnerIter> inner_; // exposition only
    [...]
public:
    iterator() requires default_initializable<OuterIter> = default;
    [...]
};

```

[Kona 2022-11-08; Accepted at joint LWG/SG9 session. Move to Immediate]

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 26.7.14.3 [\[range.join.iterator\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>> &&
            (is_reference_v<range_reference_t<V>> ||
             view<range_value_t<V>>)
    template<bool Const>
    struct join_view<V>::iterator {
        [...]
        optional<InnerIter> inner_ = InnerIter{};
        [...]
        iterator() requires default_initializable<OuterIter> &&
            default_initializable<InnerIter> = default;
        [...]
        constexpr decltype(auto) operator*() const { return *inner_; }
        [...]
        friend constexpr decltype(auto) iter_move(const iterator& i)
        noexcept(noexcept(ranges::iter_move(*i.inner_))) {
            return ranges::iter_move(*i.inner_);
        }

        friend constexpr void iter_swap(const iterator& x, const iterator& y)
        noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
        requires indirectly_swappable<InnerIter>;
    };
}

[...]

constexpr void satisfy(); // exposition only

```

-5- Effects: Equivalent to:

```

auto update_inner = [this](const iterator_t<Base>& x) -> auto&& {
    [...]
};
for (; outer_ != ranges::end(parent_>base_); ++outer_) {
    auto&& inner = update_inner(*outer_);
    inner_ = ranges::begin(inner);
    if (*inner_ != ranges::end(inner))
        return;
}
if constexpr (ref-is-glvalue)
    inner_.reset() = InnerIter{};

```

[...]

```
constexpr InnerIter operator->() const
    requires has_arrow<InnerIter> && copyable<InnerIter>;
```

-8- *Effects*: Equivalent to: return `*inner_;`

```
constexpr iterator& operator++();
```

-9- Let *inner-range* be:

[...]

-10- *Effects*: Equivalent to:

```
auto&& inner_rng = inner-range;
if (++*inner_ == ranges::end(inner_rng)) {
    ++outer_;
    satisfy();
}
return *this;
```

[...]

```
constexpr iterator& operator--()
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

-13- *Effects*: Equivalent to:

```
if (outer_ == ranges::end(parent_>base_))
    inner_ = ranges::end(*--outer_);
while (*inner_ == ranges::begin(*outer_))
    *inner_ = ranges::end(*--outer_);
--*inner_;
return *this;
```

[...]

```
friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
    requires indirectly_swappable<InnerIter>;
```

-16- *Effects*: Equivalent to: return `ranges::iter_swap(*x.inner_, *y.inner_);`

3717. `common_view::end` should improve `random_access_range` case

Section: 26.7.19.2 [[range.common.view](#)] **Status:** [Immediate](#) **Submitter:** Hewill Kang **Opened:** 2022-06-15 **Last modified:** 2022-11-11

Priority: 3

View all other [issues](#) in [`range.common.view`].

Discussion:

This issue is part of NB comment [US 47-109](#) 26 [ranges] Resolve open issues

In view of the fact that random access iterators are only required to work with its difference type, [P2393R1](#) improves the wording of `take_view`, which first convert the integer type to difference type and then operate with the iterator. However, the paper omits the handling of random access iterators in `common_view::end`, which directly operates on the return types of `ranges::begin` and `ranges::size`. We should improve this, too.

[2022-07-06; Reflector poll]

Set priority to 3 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 26.7.19.2 [\[range.common.view\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
    requires (!common_range<V> && copyable<iterator_t<V>>())
    class common_view : public view_interface<common_view<V>> {
    private:
        V base_ = V(); // exposition only
    public:
        [...]
        constexpr auto begin() {
            if constexpr (random_access_range<V> && sized_range<V>)
                return ranges::begin(base_);
            else
                return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::begin(base_));
        }

        constexpr auto begin() const requires range<const V> {
            if constexpr (random_access_range<const V> && sized_range<const V>)
                return ranges::begin(base_);
            else
                return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::begin(base_));
        }

        constexpr auto end() {
            if constexpr (random_access_range<V> && sized_range<V>)
                return ranges::begin(base_) + range_difference_t<V>(size().)ranges::size(base_);
            else
                return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::end(base_));
        }

        constexpr auto end() const requires range<const V> {
            if constexpr (random_access_range<const V> && sized_range<const V>)
                return ranges::begin(base_) + range_difference_t<const V>(size().)ranges::size(base_);
            else
                return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::end(base_));
        }

        constexpr auto size() requires sized_range<V> {
            return ranges::size(base_);
        }
        constexpr auto size() const requires sized_range<const V> {
            return ranges::size(base_);
        }
    };
    [...]
}
```

[Kona 2022-11-08; Discussed at joint LWG/SG9 session. Move to Open]

[2022-11-09 Tim updates wording per LWG discussion]

[Kona 2022-11-10; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 26.7.19.2 [\[range.common.view\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
    requires (!common_range<V> && copyable<iterator_t<V>>())
    class common_view : public view_interface<common_view<V>> {
    private:
        V base_ = V(); // exposition only
    public:
        [...]
        constexpr auto begin() {
            if constexpr (random_access_range<V> && sized_range<V>)
                return ranges::begin(base_);
            else
                return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::begin(base_));
        }

        constexpr auto begin() const requires range<const V> {
            if constexpr (random_access_range<const V> && sized_range<const V>)
                return ranges::begin(base_);
            else
                return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::begin(base_));
        }

        constexpr auto size() requires sized_range<V> {
            return ranges::size(base_);
        }
        constexpr auto size() const requires sized_range<const V> {
            return ranges::size(base_);
        }
    };
    [...]
}
```

```

        return ranges::begin(base_);
    else
        return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::begin(base_));
    }

constexpr auto end() {
    if constexpr (random_access_range<V> && sized_range<V>)
        return ranges::begin(base_) + ranges::sizedistance(base_);
    else
        return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::end(base_));
    }

constexpr auto end() const requires range<const V> {
    if constexpr (random_access_range<const V> && sized_range<const V>)
        return ranges::begin(base_) + ranges::sizedistance(base_);
    else
        return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::end(base_));
    }

constexpr auto size() requires sized_range<V> {
    return ranges::size(base_);
}
constexpr auto size() const requires sized_range<const V> {
    return ranges::size(base_);
}
};
[...]
```

[3737](#). `take_view::sentinel` should provide operator-

Section: 26.7.10.3 [\[range.take.sentinel\]](#) **Status:** [Immediate](#) **Submitter:** Hewill Kang **Opened:** 2022-07-15 **Last modified:** 2022-11-11

Priority: 3

View all other [issues](#) in `[range.take.sentinel]`.

Discussion:

This issue is part of NB comment [US 47-109](#) 26 `[ranges]` Resolve open issues

When the underlying range is not a `sized_range`, the `begin` and `end` functions of `take_view` return `counted_iterator` and `take_view::sentinel` respectively. However, the sentinel type of the underlying range may still model `sized_sentinel_for` for its iterator type, and since `take_view::sentinel` can only be compared to `counted_iterator`, this makes `take_view` no longer able to compute the distance between its iterator and sentinel.

We are needlessly losing functionality here. Since calculating the distance, in this case, is still simple, i.e. we just need to compute the minimum of `counted_iterator::count` and the difference between the underlying iterator and sentinel, I think providing operator- for `take_view::sentinel` does bring some value.

[2022-08-23; Reflector poll]

Set priority to 3 after reflector poll.

Some P0 votes, but with objections: "This seems like a) a feature not a bug - of fairly limited utility?, and b) I'd like to see an implementation (maybe it's in MSVC?) to be sure there isn't a negative interaction we're not thinking of."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 26.7.10.3 [\[range.take.sentinel\]](#), class template `take_view::sentinel` synopsis, as indicated:

```

namespace std::ranges {
    template<view V>
    template<bool Const>
    class take_view<V>::sentinel {
    private:
        using Base = maybe_const<Const, V>;                                // exposition only
        template<bool OtherConst>
        using CI = counted_iterator<iterator_t<maybe_const<OtherConst, V>>>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();                        // exposition only
    };
}
```

```

public:
    [...]
    friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

    template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);

    friend constexpr range difference t<Base>
        operator-(const sentinel& x, const CI<Const>& y)
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    template<bool OtherConst = !Const>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range difference t<maybe-const<OtherConst, V>>
        operator-(const sentinel& x, const CI<OtherConst>& y);

    friend constexpr range difference t<Base>
        operator-(const CI<Const>& x, const sentinel& y)
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    template<bool OtherConst = !Const>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range difference t<maybe-const<OtherConst, V>>
        operator-(const CI<OtherConst>& x, const sentinel& y);
    };
}

[...]
```

friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

template<bool OtherConst = !Const>
 requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
 friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);

-4- *Effects:* Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

```

friend constexpr range difference t<Base>
    operator-(const sentinel& x, const CI<Const>& y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

template<bool OtherConst = !Const>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range difference t<maybe-const<OtherConst, V>>
    operator-(const sentinel& x, const CI<OtherConst>& y);

-?- Effects: Equivalent to: return ranges::min(y.count(), x.end - y.base());
```

```

friend constexpr range difference t<Base>
    operator-(const CI<Const>& x, const sentinel& y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

template<bool OtherConst = !Const>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range difference t<maybe-const<OtherConst, V>>
    operator-(const CI<OtherConst>& x, const sentinel& y);

-?- Effects: Equivalent to: return -(y - x);
```

[Kona 2022-11-08; Discussed at joint LWG/SG9 session. Move to Open]

[2022-11-09 Tim updates wording following LWG discussion]

This case is only possible if the source view is not a `sized_range`, yet its iterator/sentinel types model `sized_sentinel_for` (typically when source is an input range). In such a case we should just have begin compute the correct size so that end can just return `default_sentinel`.

[Kona 2022-11-10; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 26.7.10.2 [\[range.take.view\]](#), class template `take_view` synopsis, as indicated:

```
namespace std::ranges {
    template<view V>
    class take_view : public view_interface<take_view<V>> {
    private:
        [...]
    public:
        [...]
        constexpr auto begin() requires (!simple-view<V>) {
            if constexpr (sized_range<V>) {
                if constexpr (random_access_range<V>) {
                    return ranges::begin(base_);
                } else {
                    auto sz = range_difference_t<V>(size());
                    return counted_iterator(ranges::begin(base_), sz);
                }
            } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>)_ {
                auto it = ranges::begin(base_);
                auto sz = std::min(count_, ranges::end(base_) - it);
                return counted_iterator(std::move(it), sz);
            } else {
                return counted_iterator(ranges::begin(base_), count_);
            }
        }

        constexpr auto begin() const requires range<const V> {
            if constexpr (sized_range<const V>) {
                if constexpr (random_access_range<const V>) {
                    return ranges::begin(base_);
                } else {
                    auto sz = range_difference_t<const V>(size());
                    return counted_iterator(ranges::begin(base_), sz);
                }
            } else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>)_ {
                auto it = ranges::begin(base_);
                auto sz = std::min(count_, ranges::end(base_) - it);
                return counted_iterator(std::move(it), sz);
            } else {
                return counted_iterator(ranges::begin(base_), count_);
            }
        }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (sized_range<V>) {
                if constexpr (random_access_range<V>)
                    return ranges::begin(base_) + range_difference_t<V>(size());
                else
                    return default_sentinel;
            } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>)_ {
                return default_sentinel;
            } else {
                return sentinel<false>{ranges::end(base_)};
            }
        }

        constexpr auto end() const requires range<const V> {
            if constexpr (sized_range<const V>) {
                if constexpr (random_access_range<const V>)
                    return ranges::begin(base_) + range_difference_t<const V>(size());
                else
                    return default_sentinel;
            } else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>)_ {
                return default_sentinel;
            } else {
                return sentinel<true>{ranges::end(base_)};
            }
        }
        [...]
    };
    [...]
}
```

[3753](#). Clarify entity vs. freestanding entity

Section: 16.3.3.6 [\[freestanding.entity\]](#) Status: [Immediate](#) Submitter: Ben Craig Opened: 2022-08-23 Last modified: 2022-11-11

Priority: 2

View other [active issues](#) in [freestanding.entity].

View all other [issues](#) in [freestanding.entity].

Discussion:

This addresses NB comment [GB-075](#) ([freestanding.entity] "Freestanding entities" are not entities)

16.3.3.6 [\[freestanding.entity\]](#) p1 defines a freestanding entity as a declaration or macro definition.

16.3.3.6 [\[freestanding.entity\]](#) p3 then says "entities followed with a comment [...] are freestanding entities".

This is inconsistent, and breaks with macros, because macros are not entities, but they can be freestanding entities.

[2022-09-23; Reflector poll]

Set priority to 2 after reflector poll.

It's confusing for "freestanding entities" to be two things, neither of which are entities. Declarations may declare entities, they are not entities themselves. Given this definition, p6/7/8 makes no sense. A namespace can't be a freestanding entity since it's neither a declaration nor a macro definition.

"freestanding entities" is not best name, given the collision with core entity, but I think that this is separable issue.

[2022-09-28; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to the forthcoming C++23 CD.

- Modify 16.3.3.6 [\[freestanding.entity\]](#) as indicated:

-3- In a header synopsis, **entities** **declarations and macro definitions** followed with a comment that includes *freestanding* are freestanding entities.

[2022-11-06; Ben Craig provides new wording]

[2022-11-07; Kona - move to open]

New proposed resolution to be added.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 16.3.3.6 [\[freestanding.entity\]](#) as indicated:

[Drafting note: Replace the section name [freestanding.entity] by [freestanding.item] throughout the working draft]

16.3.3.6 Freestanding **entities **items** [freestanding.item **entity**]**

-1- A *freestanding* **entity** **item** is a **declaration** **an entity** or macro definition that is present in a freestanding implementation and a hosted implementation.

-2- Unless otherwise specified, the requirements on freestanding **entities** **items** on a freestanding implementation are the same as the corresponding requirements in a hosted implementation.

-3- In a header synopsis, entities **introduced by declarations** followed with a comment that includes *freestanding* are freestanding **entities** **items**.

-?- In a header synopsis, macro definitions followed with a comment that includes *freestanding* are freestanding items.

[...]

-4- If a header synopsis begins with a comment that includes *all freestanding*, then all of the **entities introduced by** declarations ~~and macro definitions~~ in the header synopsis are freestanding **entities items**.

-?- If a header synopsis begins with a comment that includes *all freestanding*, then all of the macro definitions in the header synopsis are freestanding items.

[...]

-5- Deduction guides for freestanding **entity item** class templates are freestanding **entities items**.

-6- Enclosing namespaces of freestanding **entities items** are freestanding **entities items**.

-7- Friends of freestanding **entities items** are freestanding **entities items**.

-8- Entities denoted by freestanding **entity item** *typedef-names* and freestanding **entity item** alias templates are freestanding **entities items**.

2. Modify 16.4.2.5 [\[compliance\]](#) as indicated:

-3- For each of the headers listed in Table 28, a freestanding implementation provides at least the freestanding **entities items** (16.3.3.6 [\[freestanding.entity\]](#)) declared in the header.

3. Modify 22.10.15.5 [\[func.bind.place\]](#) as indicated:

-3- Placeholders are freestanding **entities items** (16.3.3.6 [\[freestanding.entity\]](#)).

[2022-11-08; Ben Craig provides improved wording]

This combined resolution addresses both 3753 and LWG [3815](#), and has already been reviewed by LWG.

This resolves ballot comment GB-75. It also partially addresses GB-130 (along with LWG [3814](#)).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 16.3.3.6 [\[freestanding.entity\]](#) as indicated:

[Drafting note: Replace the section name `[freestanding.entity]` by `[freestanding.item]` throughout the working draft]

16.3.3.6 Freestanding **entities items** [\[freestanding.item entity\]](#)

-1- A *freestanding* **entity item** is ~~a declaration~~ **an entity** or macro definition that is present in a freestanding implementation and a hosted implementation.

-2- Unless otherwise specified, the requirements on **non-namespace** freestanding **entities items** on a freestanding implementation are the same as the corresponding requirements in a hosted implementation.

[Note: Enumerators impose requirements on their enumerations. Freestanding item enumerations have the same enumerators on freestanding implementations and hosted implementations. Members and deduction guides impose requirements on their class types. Class types have the same deduction guides and members on freestanding implementations and hosted implementations. — end note]

-3- In a header synopsis, ~~entities~~ **each entity introduced by a declaration** followed ~~with~~ **by** a comment that includes *freestanding* ~~are is a~~ freestanding **entities item**.

-?- In a header synopsis, each macro definition followed by a comment that includes *freestanding* is a freestanding item.

[...]

-4- If a header synopsis begins with a comment that includes *all freestanding*, then ~~all of the declarations and macro definitions~~ each entity introduced by a declaration in the header synopsis ~~are~~ is a freestanding ~~entities~~ item.

-2- If a header synopsis begins with a comment that includes *all freestanding*, then each macro definition in the header synopsis is a freestanding item.

[...]

~~-5- Deduction guides for freestanding entity class templates are freestanding entities.~~

~~-6- Enclosing namespaces of freestanding entities are freestanding entities.~~ Each enclosing namespace of each freestanding item is a freestanding item.

~~-7- Friends of freestanding entities are freestanding entities.~~ Each friend of each freestanding item is a freestanding item.

~~-8- Entities denoted by freestanding entity typedef names and freestanding entity alias templates are freestanding entities.~~ Each entity denoted by each freestanding item typedef name and each freestanding item alias template is a freestanding item.

2. Modify 16.4.2.5 [\[compliance\]](#) as indicated:

-3- For each of the headers listed in Table 28, a freestanding implementation provides at least the freestanding ~~entities~~ items (16.3.3.6 [\[freestanding.entity.item\]](#)) declared in the header.

3. Modify 22.10.15.5 [\[func.bind.place\]](#) as indicated:

-3- Placeholders are freestanding ~~entities~~ items (16.3.3.6 [\[freestanding.entity.item\]](#)).

[2022-11-09; Ben Craig and Tomasz provide improved wording]

This new resolution merges definition of freestanding item for entity in macro into bullet lists. It still addresses both 3753 and LWG [3815](#).

This resolves ballot comment GB-75. It also partially addresses GB-130 (along with LWG [3814](#)).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 16.3.3.6 [\[freestanding.entity\]](#) as indicated:

[Drafting note: Replace the section name [freestanding.entity] by [freestanding.item] throughout the working draft]

16.3.3.6 Freestanding ~~entities~~ items ~~[freestanding.entity]~~

-1- A freestanding ~~entity~~ item is ~~a declaration~~ an entity or macro definition that is present in a freestanding implementation and a hosted implementation.

-2- Unless otherwise specified, the requirements on freestanding ~~entities~~ items, ~~except namespaces~~, ~~on~~ for a freestanding implementation are the same as the corresponding requirements ~~in~~ for a hosted implementation.

[Note: This implies that freestanding item enumerations have the same enumerators on freestanding implementations and hosted implementations. Furthermore, class types have the same deduction guides and members on freestanding implementations and hosted implementations. — end note]

-3- An entity is a freestanding item, if it is:

(3.1) — introduced by a declaration in the header synopsis, and

(3.1.1) — the declaration is followed by a comment that includes *freestanding*, or

(3.1.2) — the header synopsis begins with a comment that includes *all freestanding*;

(3.2) — an enclosing namespace of a freestanding item,

(3.3) — a friend of a freestanding item,

(3.4) — denoted by a *typedef-name*, that is a freestanding item, or

(3.5) — denoted by a template alias, that is a freestanding item.

~~-4- A macro definition is a freestanding item, if it is defined in the header synopsis and~~

~~(4.1) — the definition is followed by a comment that includes *freestanding* in the header synopsis, or~~

~~(4.2) — the header synopsis begins with a comment that includes *all freestanding*.~~

~~-3- In a header synopsis, entities followed with a comment that includes *freestanding* are freestanding entities.~~

[Example 1: ... — end example]

~~-4- If a header synopsis begins with a comment that includes *all freestanding*, then all of the declarations and macro definitions in the header synopsis are freestanding entities.~~

[Example 2: ... — end example]

~~-5- Deduction guides for freestanding entity class templates are freestanding entities.~~

~~-6- Enclosing namespaces of freestanding entities are freestanding entities.~~

~~-7- Friends of freestanding entities are freestanding entities.~~

~~-8- Entities denoted by freestanding entity *typedef-names* and freestanding entity alias templates are freestanding entities.~~

2. Modify 16.4.2.5 [compliance] as indicated:

~~-3- For each of the headers listed in Table 28, a freestanding implementation provides at least the freestanding entities~~ **items** ~~(16.3.3.6 [freestanding.entity~~ **item** ~~]) declared in the header.~~

3. Modify 22.10.15.5 [func.bind.place] as indicated:

~~-3- Placeholders are freestanding entities~~ **items** ~~(16.3.3.6 [freestanding.entity~~ **item** ~~]).~~

[2022-11-10; Tomasz provide improved wording]

Updated wording to support freestanding *typedef-names* and using declaration that are not entities. It still addresses both 3753 and LWG [3815](#).

This resolves ballot comment GB-75. It also partially addresses GB-130 (along with LWG [3814](#)).

[Kona 2022-11-10; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 16.3.3.6 [freestanding.entity] as indicated:

[Drafting note: Replace the section name [freestanding.entity] by [freestanding.item] throughout the working draft]

16.3.3.6 Freestanding ~~entities~~ **items** ~~[freestanding.entity]~~

-1- A freestanding ~~entity~~ **item** is a declaration, ~~entity~~, *typedef-name*, or macro ~~definition~~ that is **required to be** present in a freestanding implementation and a hosted implementation.

-2- Unless otherwise specified, the requirements on freestanding ~~entities~~ ~~items~~ ~~on~~ ~~for~~ a freestanding implementation are the same as the corresponding requirements ~~in~~ ~~for~~ a hosted implementation, except that not all of the members of the namespaces are required to be present.

[Note: This implies that freestanding item enumerations have the same enumerators on freestanding implementations and hosted implementations. Furthermore, class types have the same members and class templates have the same deduction guides on freestanding implementations and hosted implementations. — end note]

-3- A declaration in a header synopsis is a freestanding item if

(3.1) — it is followed by a comment that includes *freestanding*, or

(3.1) — the header synopsis begins with a comment that includes *all freestanding*.

-4- An entity or *typedef-name* is a freestanding item if it is:

(4.1) — introduced by a declaration that is a freestanding item,

(4.2) — an enclosing namespace of a freestanding item,

(4.3) — a friend of a freestanding item,

(4.4) — denoted by a *typedef-name* that is a freestanding item, or

(4.5) — denoted by an alias template that is a freestanding item.

-5- A macro is a freestanding item if it is defined in a header synopsis and

(5.1) — the definition is followed by a comment that includes *freestanding*, or

(5.2) — the header synopsis begins with a comment that includes *all freestanding*.

~~-3- In a header synopsis, entities followed with a comment that includes *freestanding* are freestanding entities.~~

[Example 1: ... — end example]

~~-4- If a header synopsis begins with a comment that includes *all freestanding*, then all of the declarations and macro definitions in the header synopsis are freestanding entities..~~

[Example 2: ... — end example]

~~-5- Deduction guides for freestanding entity class templates are freestanding entities.~~

~~-6- Enclosing namespaces of freestanding entities are freestanding entities.~~

~~-7- Friends of freestanding entities are freestanding entities.~~

~~-8- Entities denoted by freestanding entity *typedef-names* and freestanding entity alias templates are freestanding entities.~~

2. Modify 16.4.2.5 [\[compliance\]](#) as indicated:

-3- For each of the headers listed in Table 28, a freestanding implementation provides at least the freestanding ~~entities~~ ~~items~~ (16.3.3.6 [\[freestanding.entity.item\]](#)) declared in the header.

3. Modify 22.10.15.5 [\[func.bind.place\]](#) as indicated:

-3- Placeholders are freestanding ~~entities~~ ~~items~~ (16.3.3.6 [\[freestanding.entity.item\]](#)).

3766. `view_interface::cbegin` is underconstrained

Section: 26.5.3.1 [\[view.interface.general\]](#) Status: [Immediate](#) Submitter: Hewill Kang Opened: 2022-09-04 Last modified: 2022-11-11

Priority: 2

View other [active issues](#) in [\[view.interface.general\]](#).

View all other [issues](#) in [\[view.interface.general\]](#).

Discussion:

Currently, `view_interface::cbegin` simply returns `ranges::cbegin(derived())`, which returns the type alias `const_iterator` for its iterator, which requires that the template parameter `I` must model the `input_iterator`.

Given that `view_interface::cbegin` does not have any constraints, when `D` models only `output_range`, calling its `cbegin()` will result in a hard error inside the function body:

```
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v;
    auto r = std::views::counted(std::back_inserter(v), 3);
    auto b = r.cbegin(); // hard error
}
```

We should add a constraint for `view_interface::cbegin` that `D` must model `input_range`.

[2022-09-23; Reflector poll]

Set priority to 2 after reflector poll.

This should be done for `cend` too.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 26.5.3.1 [\[view.interface.general\]](#) as indicated:

```
namespace std::ranges {
    template<class D>
        requires is_class_v<D> && same_as<D, remove_cv_t<D>>
    class view_interface {
        [...]
    public:
        [...]
        constexpr auto cbegin() requires input_range<D> {
            return ranges::cbegin(derived());
        }
        constexpr auto cbegin() const requires input_range<const D> {
            return ranges::cbegin(derived());
        }
        [...]
    };
}
```

[2022-09-25; Hewill provides improved wording]

[Kona 2022-11-08; Accepted at joint LWG/SG9 session. Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 26.5.3.1 [\[view.interface.general\]](#) as indicated:

```
namespace std::ranges {
    template<class D>
        requires is_class_v<D> && same_as<D, remove_cv_t<D>>
    class view_interface {
        [...]
    public:
        [...]
        constexpr auto cbegin() requires input_range<D> {
            return ranges::cbegin(derived());
        }
        constexpr auto cbegin() const requires input_range<const D> {
            return ranges::cbegin(derived());
        }
        constexpr auto cend() requires input_range<D> {
            return ranges::cend(derived());
        }
        constexpr auto cend() const requires input_range<const D> {
            return ranges::cend(derived());
        }
    };
}
```

```
    }  
    [...]  
};  
}
```

3814. Add freestanding items requested by NB comments

Section: 20.2.2 [\[memory.syn\]](#), 26.2 [\[ranges.syn\]](#), 33.5.2 [\[atomics.syn\]](#) **Status:** [Immediate](#) **Submitter:** Ben Craig **Opened:** 2022-11-06 **Last modified:** 2022-11-11

Priority: Not Prioritized

View all other [issues](#) in [\[memory.syn\]](#).

Discussion:

This addresses the following NB comments:

- [GB-085](#) (20.2.2 [\[memory.syn\]](#) `start_lifetime_as` and `start_lifetime_as_array` should be freestanding)
- [GB-110](#) (26.2 [\[ranges.syn\]](#) New views should be freestanding (repeat, stride, cartesian_product))
- (partial) [GB-130](#) (33.5.2 [\[atomics.syn\]](#) `memory_order_acquire` etc should be freestanding)

The explicit lifetime management functions requested by GB-085 have not been reviewed by LEWG in the context of freestanding, but they seem non-controversial in that context. None of the requested lifetime management functions run any code. I believe these were missed in post-merge conflict searches because the papers weren't targeted to LEWG or LWG at the time of those searches.

The ranges facilities requested by GB-110 *have* been reviewed on the LEWG mailing list in the context of freestanding. [P1642R11](#) mentions the repeat, stride, and cartesian_product papers in "Potential Post-LEWG merge conflicts". All were discussed in an April 2022 reflector discussion and received six votes in favor of allowing these papers into freestanding, with no opposition.

The atomics facilities requested by GB-130 are essentially new names for existing facilities. Marking these as freestanding isn't concerning. There are concerns in GB-130 dealing with the specification details of freestanding enums, but those concerns won't be addressed in this issue.

[Kona 2022-11-07; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 20.2.2 [\[memory.syn\]](#), header `<memory>` synopsis, as indicated:

```
[...]  
// 20.2.6 \[obj.lifetime\], explicit lifetime management  
template<class T>  
    T* start_lifetime_as(void* p) noexcept; // freestanding  
template<class T>  
    const T* start_lifetime_as(const void* p) noexcept; // freestanding  
template<class T>  
    volatile T* start_lifetime_as(volatile void* p) noexcept; // freestanding  
template<class T>  
    const volatile T* start_lifetime_as(const volatile void* p) noexcept; // freestanding  
template<class T>  
    T* start_lifetime_as_array(void* p, size_t n) noexcept; // freestanding  
template<class T>  
    const T* start_lifetime_as_array(const void* p, size_t n) noexcept; // freestanding  
template<class T>  
    volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept; // freestanding  
template<class T>  
    const volatile T* start_lifetime_as_array(const volatile void* p, size_t n) noexcept; // freestanding  
[...]
```

2. Modify 26.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
[...]  
// 26.6.5 \[range.repeat\], repeat view  
template<move_constructible W, semiregular Bound = unreachable_sentinel_t>  
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>>  
        && (is_integer_like<Bound> || same_as<Bound, unreachable_sentinel_t>))  
    class repeat_view; // freestanding
```

```

namespace views { inline constexpr unspecified repeat = unspecified; } // freestanding
[...]
// 26.7.30 [range.stride], stride view
template<input_range V>
    requires view<V>
class stride_view; // freestanding

template<class V>
    inline constexpr bool enable_borrowed_range<stride_view<V>> = enable_borrowed_range<V>; // freestanding

namespace views { inline constexpr unspecified stride = unspecified; } // freestanding

// 26.7.31 [range.cartesian], cartesian product view
template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
class cartesian_product_view; // freestanding

namespace views { inline constexpr unspecified cartesian_product = unspecified; } // freestanding
[...]
```

3. Modify 33.5.2 [atomics.syn], header <atomic> synopsis, as indicated:

```

namespace std {
    // 33.5.4 [atomics.order], order and consistency
    enum class memory_order : unspecified; // freestanding
    inline constexpr memory_order memory_order_relaxed = memory_order::relaxed; // freestanding
    inline constexpr memory_order memory_order_consume = memory_order::consume; // freestanding
    inline constexpr memory_order memory_order_acquire = memory_order::acquire; // freestanding
    inline constexpr memory_order memory_order_release = memory_order::release; // freestanding
    inline constexpr memory_order memory_order_acq_rel = memory_order::acq_rel; // freestanding
    inline constexpr memory_order memory_order_seq_cst = memory_order::seq_cst; // freestanding
    [...]
}
```

3816. flat_map and flat_multimap should impose sequence container requirements

Section: 24.6.9.1 [flat.map.overview], 24.6.10.1 [flat.multimap.overview] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2022-11-08 **Last modified:** 2022-11-09

Priority: Not Prioritized

Discussion:

This is resolution of [US 42-103](#) (24.6.9.1 [flat.map.overview] p6 24.6.10.1 [flat.multimap.overview] p6 Clearing when restoring invariants).

Currently both 24.6.9.1 [flat.map.overview] p7 and 24.6.10.1 [flat.multimap.overview] p7 claims that flat_(multi)map supports "Any sequence container (24.2.4 [sequence.reqmts]) C supporting Cpp17RandomAccessIterator", which arguably includes std::array (see LWG 617). This is incorrect as std::array does not provide operations required to restore these adaptors invariant, including clear. We should require that C meets sequence container requirements, and we state that fact explicitly in 24.3.7.1 [array.overview] p3: "An array meets some of the requirements of a sequence container (24.2.4 [sequence.reqmts])".

[Kona 2022-11-08; Move to Immediate status]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 24.6.9.1 [flat.map.overview] as indicated:

-7- Any ~~sequence container (24.2.4 [sequence.reqmts])~~ type C ~~supporting Cpp17RandomAccessIterator~~ that meets sequence container requirements (24.2.4 [sequence.reqmts]) can be used to instantiate flat_map, as long as C::iterator meets the Cpp17RandomAccessIterator requirements and invocations of member functions C::size and C::max_size do not exit via an exception. In particular, vector (24.3.11 [vector]) and deque (24.3.8 [deque]) can be used.

2. Modify 24.6.10.1 [flat.multimap.overview] as indicated:

-7- Any ~~sequence container (24.2.4 [sequence.reqmts])~~ type C ~~supporting Cpp17RandomAccessIterator~~ that meets sequence container requirements (24.2.4 [sequence.reqmts]) can be used to instantiate flat_multimap, as long as C::iterator meets the Cpp17RandomAccessIterator requirements and invocations of member functions C::size and C::max_size do not exit via an exception. In particular, vector (24.3.11 [vector]) and deque (24.3.8 [deque]) can be used.

3817. Missing preconditions on forward_list modifiers

Section: 24.3.9.5 [\[forward.list.modifiers\]](#) Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2022-11-08 Last modified: 2022-11-12

Priority: Not Prioritized

Discussion:

This is resolution of [GB-101](#) (24.3.9.5 [\[forward.list.modifiers\]](#) p12,15,20,21 Missing preconditions on forward_list modifiers).

Some of the modifiers to forward_list are special to that container and accordingly are not described in 24.2 [\[container.requirements\]](#). Specifically, insert_after (iterator overload), insert_range_after and emplace_after do not verify that the value_type is Cpp17EmplaceConstructible from the appropriate argument(s). Furthermore insert_after (value overloads) are missing Cpp17CopyInsertable/Cpp17MoveInsertable requirements.

[Kona 2022-11-08; Move to Ready]

[Kona 2022-11-12; Correct status to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 24.3.9.5 [\[forward.list.modifiers\]](#) as indicated:

[Drafting note: emplace_front, push_front, and prepend_range are already covered by 24.2 [\[container.requirements\]](#).]

```
iterator insert_after(const_iterator position, const T& x);
```

Preconditions: `T` is Cpp17CopyInsertable into forward_list. position is before begin() or is a dereferenceable iterator in the range [begin(), end()).

Effects: Inserts a copy of `x` after position.

Returns: An iterator pointing to the copy of `x`.

```
iterator insert_after(const_iterator position, T&& x);
```

Preconditions: `T` is Cpp17MoveInsertable into forward_list. position is before_begin() or is a dereferenceable iterator in the range [begin(), end()).

Effects: Inserts a copy of `x` after position.

Returns: An iterator pointing to the copy of `x`.

```
iterator insert_after(const_iterator position, size_type n, const T& x);
```

Preconditions: `T` is Cpp17CopyInsertable into forward_list. position is before_begin() or is a dereferenceable iterator in the range [begin(), end()).

Effects: Inserts `n` copies of `x` after position.

Returns: An iterator pointing to the last inserted copy of `x`, or position if `n == 0` is true.

```
template<class InputIterator>
```

```
iterator insert_after(const_iterator position, InputIterator first, InputIterator last);
```

Preconditions: `T` is Cpp17EmplaceConstructible into forward_list from `*first`. position is before_begin() or is a dereferenceable iterator in the range [begin(), end()). Neither first nor last are iterators in `*this`.

Effects: Inserts copies of elements in [first, last) after position.

Returns: An iterator pointing to the last inserted element, or position if first == last is true.

```
template<container-compatible-range<T> R>
```

```
iterator insert_after(const_iterator position, R&& rg);
```

-15- *Preconditions:* `T is Cpp17EmplaceConstructible` into forward list from `*ranges::begin(rg)`. position is before `begin()` or is a dereferenceable iterator in the range `[begin(), end())`. `rg` and `*this` do not overlap.

-16- *Effects:* Inserts copies of elements in range `rg` after position.

-17- *Returns:* An iterator pointing to the last inserted element, or position if `rg` is empty.

```
iterator insert_after(const_iterator position, initializer_list<T> il);
```

-18- *Effects:* `Equivalent to: return` `insert_after(position, il.begin(), il.end())`;

~~-19- *Returns:* An iterator pointing to the last inserted element or position if `il` is empty.~~

```
template<class... Args>
iterator emplace_after(const_iterator position, Args&&... args);
```

-20- *Preconditions:* `T is Cpp17EmplaceConstructible` into forward list from `std::forward<Args>(args)...`. position is before `begin()` or is a dereferenceable iterator in the range `[begin(), end())`.

-21- *Effects:* Inserts an object of type `value_type` ~~constructed~~ `direct-non-list-initialized` with ~~`value_type{std::forward<Args>(args)...`~~ after position.

-22- *Returns:* An iterator pointing to the new object.

3818. Exposition-only concepts are not described in library intro

Section: 16.3.3 [\[conventions\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2022-11-08 **Last modified:** 2022-11-12

Priority: Not Prioritized

Discussion:

This is the resolution for [GB-074](#).

The comment is:

16.3.3.2 [\[expos.only.func\]](#) introduces exposition-only function templates. 16.3.3.2 [\[expos.only.types\]](#) introduces exposition-only types. 16.3.3.5 [\[objects.within.classes\]](#) introduces exposition-only private members.

There is nothing about exposition-only concepts, despite them being used extensively in the library clauses.

The same problem exists for exposition-only variable templates.

[Kona 2022-11-08; Move to Immediate status]

Previous resolution [SUPERSEDED]:

1. Modify 16.3.3.2 [\[expos.only.func\]](#) as indicated, changing the stable name:

16.3.3.2 Exposition-only ~~function templates~~ `entities` [\[expos.only.funcentity\]](#)

-1- Several ~~function templates~~ `entities` defined in 17 [\[support\]](#) through 33 [\[thread\]](#) and D [\[depr\]](#) are only defined for the purpose of exposition. The declaration of such a ~~function~~ `entity` is followed by a comment ending in *exposition only*.

2. Strike 16.3.3.3.2 [\[expos.only.types\]](#) as redundant:

~~16.3.3.3.2 Exposition-only types [\[expos.only.types\]](#)~~

~~-1- Several types defined in 17 [\[support\]](#) through 33 [\[thread\]](#) and D [\[depr\]](#) are defined for the purpose of exposition. The declaration of such a type is followed by a comment ending in *exposition only*.~~

~~*Example 1:*~~

```
namespace std {
extern "C" using some_handler = int(int, void*, double); // exposition only
}
```

~~The type placeholder `some_handler` can now be used to specify a function that takes a callback parameter with C language linkage. — end example~~

[2022-11-09 Tim reopens]

During LWG review of [3753](#), it was pointed out that `typedef-names` are not necessarily entities.

[Kona 2022-11-11; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 16.3.3.2 [\[expos.only.func\]](#) as indicated, changing the stable name:

16.3.3.2 Exposition-only ~~functions~~**entities, etc.** ~~[expos.only.func]~~**[entity]**

-1- Several ~~function templates~~**entities and typedef-names** defined in 17 [\[support\]](#) through 33 [\[thread\]](#) and D [\[depr\]](#) are only defined for the purpose of exposition. The declaration of such ~~a function~~**an entity or typedef-name** is followed by a comment ending in *exposition only*.

2. Strike 16.3.3.3.2 [\[expos.only.types\]](#) as redundant:

~~16.3.3.3.2 Exposition-only types~~ **[expos.only.types]**

~~-1- Several types defined in 17 [\[support\]](#) through 33 [\[thread\]](#) and D [\[depr\]](#) are defined for the purpose of exposition. The declaration of such a type is followed by a comment ending in *exposition only*.~~

~~[Example 1:]~~

```
namespace std {  
extern "C" using some_handler = int(int, void*, double); // exposition only  
}
```

~~The type placeholder `some_handler` can now be used to specify a function that takes a callback parameter with C language linkage. — end example~~

[3822](#). Avoiding normalization in `filesystem::weakly_canonical`

Section: 31.12.13.40 [\[fs.op.weakly.canonical\]](#) **Status:** [Immediate](#) **Submitter:** US **Opened:** 2022-11-08 **Last modified:** 2022-11-10

Priority: Not Prioritized

View all other [issues](#) in `[fs.op.weakly.canonical]`.

Discussion:

This addresses NB comment [US-60-125](#) (31.12.13.40 [\[fs.op.weakly.canonical\]](#) Avoiding normalization)

NB comment: "Implementations cannot avoid normalization because arbitrary file system changes may have occurred since any previous call. Proposed change: Remove the paragraph."

[Kona 2022-11-07; LWG review]

Discussion revolved around two different interpretations of the *Remarks*:

- It's normative recommendation for path objects to cache some kind of flag that indicates they are in a normalized form, which would be checked in `weakly_canonical` to avoid normalizing again. This is the interpretation assumed by the NB comment, which correctly notes that such caching would be unreliable.
- It's an optimization hint to implementors that they should not normalize the result if each component has been canonicalized one-by-one. That is, when canonicalizing `"/a/b/c"` by incrementally building up canonicalized components, if the entire path exists then the result will already have been normalized and should not be normalized again explicitly.

For the first interpretation, the recommendation is a bad recommendation and should be removed as suggested by the comment. For the second interpretation, we don't need to give hints to implementors about not doing unnecessary work; they already know they shouldn't do that. Either way, it should go.

[Kona 2022-11-09; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 31.12.13.40 [\[fs.op.weakly.canonical\]](#) as indicated:

```
path filesystem::weakly_canonical(const path& p);
path filesystem::weakly_canonical(const path& p, error_code& ec);
```

- 1- *Effects*: Using `status(p)` or `status(p, ec)`, respectively, to determine existence, return a path composed by operator `/=` from the result of calling `canonical()` with a path argument composed of the leading elements of `p` that exist, if any, followed by the elements of `p` that do not exist, if any. For the first form, `canonical()` is called without an `error_code` argument. For the second form, `canonical()` is called with `ec` as an `error_code` argument, and `path()` is returned at the first error occurrence, if any.
- 2- *Postconditions*: The returned path is in normal form (31.12.6.2 [\[fs.path.generic\]](#)).
- 3- *Returns*: `p` with symlinks resolved and the result normalized (31.12.6.2 [\[fs.path.generic\]](#)).
- 4- *Throws*: As specified in 31.12.5 [\[fs.err.report\]](#).
- 5- *Remarks*: Implementations should avoid unnecessary normalization such as when `canonical` has already been called on the entirety of `p`.

3823. Unnecessary precondition for `is_aggregate`

Section: 21.3.5.4 [\[meta.unary.prop\]](#) **Status:** [Immediate](#) **Submitter:** Tomasz Kamiński **Opened:** 2022-11-09 **Last modified:** 2022-11-10

Priority: Not Prioritized

View other [active issues](#) in [\[meta.unary.prop\]](#).

View all other [issues](#) in [\[meta.unary.prop\]](#).

Discussion:

This is resolution of [GB-090](#) (21.3.5.4 [\[meta.unary.prop\]](#) Unnecessary precondition for `is_aggregate`).

The precondition for `is_aggregate` is "`remove_all_extents_t<T>` shall be a complete type or cv void." This means that `is_aggregate_v<Incomplete[2]>` is undefined, but an array is always an aggregate, we don't need a complete element type to know that.

Historically the `is_aggregate` was introduced by LWG [2911](#) as part of the resolution of the NB comments. The comment proposed to introduce this trait with requirement "`remove_all_extents_t<T>` shall be a complete type, an array type, or (possibly cv-qualified) void.", that is close to resolution proposed in this issue. According to notes this was simplified during review, after realizing that `remove_all_extents_t<T>` is never an array type.

[Kona 2022-11-09; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 21.3.3 [\[meta.type.synop\]](#), Table 46 ([tab:meta.unary.prop]) — "Type property predicates" — as indicated:

Table 48: Type property predicates [tab:meta.unary.prop]		
Template	Condition	Preconditions
...		
<code>template<class T,</code> <code>class U></code> <code>struct is_aggregate;</code>	<code>T</code> is an aggregate type (9.4.2 [dcl.init.aggr])	<code>remove_all_extents_t<T></code> shall be an array type , a complete type, or cv void.

3824. Number of bind placeholders is underspecified

Section: 22.10.15.5 [\[func.bind.place\]](#) **Status:** [Immediate](#) **Submitter:** Tomasz Kamiński **Opened:** 2022-11-09 **Last modified:** 2022-11-10

Priority: Not Prioritized

View all other [issues](#) in [func.bind.place].

Discussion:

This is resolution of [GB-95](#) (22.10.15.5 [\[func.bind.place\]](#) Number of bind placeholders is underspecified).

22.10.2 [\[functional.syn\]](#) and 22.10.15.5 [\[func.bind.place\]](#) both contain a comment that says "M is the implementation-defined number of placeholders". There is no wording anywhere to say that there is any such number, only that comment.

[Kona 2022-11-09; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 22.10.15.5 [\[func.bind.place\]](#) as indicated:

```
namespace std::placeholders {  
    // M is the implementation-defined number of placeholders  
    see below _1;  
    see below _2;  
    .  
    .  
    .  
    see below _M;  
}
```

-?- The number M of placeholders is implementation-defined.

-1- All placeholder types meet the *Cpp17DefaultConstructible* and *Cpp17CopyConstructible* requirements, and their default constructors and copy/move constructors are constexpr functions that do not throw exceptions. It is implementation-defined whether placeholder types meet the *Cpp17CopyAssignable* requirements, but if so, their copy assignment operators are constexpr functions that do not throw exceptions.

[3826](#). Redundant specification [for overload of `yield_value`]

Section: 26.8.5 [\[coro.generator.promise\]](#) **Status:** [Immediate](#) **Submitter:** US **Opened:** 2022-11-10 **Last modified:** 2022-11-12

Priority: Not Prioritized

Discussion:

This is in resolution of [US 56-118](#) (26.8.5 [\[coro.generator.promise\]](#) Redundant specification).

[Paragraph 14] is redundant given [paragraphs] 13 and 12. Remove it.

Paragraphs 12 and 14 are identical: "*Remarks:* A *yield-expression* that calls this function has type void (7.6.17 [\[expr.yield\]](#)).". Paragraph 13 states that the overload of `yield_value` that accepts `ranges::elements_of` for arbitrary ranges has "*Effects:* Equivalent to:" calling the overload of `yield_value` that accepts specializations of generator, which paragraph 12 specifies. Per 16.3.2.4 [\[structure.specifications\]](#) paragraph 4, the former overload "inherits" the *Remarks* of paragraph 12 making paragraph 14 redundant.

LWG is concerned that the redundancy is not immediately obvious — it depends on an understanding of how await expressions function — so we'd like to preserve comment despite that we agree that it is normatively redundant.

[2022-11-10 Casey provides wording]

[Kona 2022-11-11; Move to Immediate]

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 26.8.5 [\[coro.generator.promise\]](#) as indicated:

```
template<ranges::input_range R, class Alloc>  
    requires convertible_to<ranges::range_reference_t<R>, yielded>  
    auto yield_value(ranges::elements_of<R, Alloc> r) noexcept;
```

-13- *Effects:* Equivalent to: [...]

-14- **Remarks:** **[Note 1:** A *yield-expression* that calls this function has type void (7.6.17 [\[expr.yield\]](#)). **— end**
note]